2's complement: The MSB 最高位 (Most Significant Bit) carries a negative weight. Range $[-2^{N-1}, 2^{N-1}-1]$. Sign extension to do operation
Fixed point number: Qm.n format: n fractional bits, N = n+m+1 bit number, MSB for 2's complement sign bit. dynamic range=6(n+m). Q0.15, Q15.0 90dB. Precision = difference between 2 consecutive numbers, $= 2^{-n}$

Dynamic range $= 20\log_{10}\left(\frac{\text{largest number}}{\text{smallest positive number excluding 0}}\right)$

Adding two N-bit Qm.n numbers require (N+1)bit Q(m +1).n format to avoid overflow. **Multiplying** two N-bit Qm.n numbers requires 2N-bit Q(2m+1).(2n) format to avoid overflow
32-bit single precision format: 31 sign bit, 23-30 8bit exponent, 0-22 23bit mantissa. Exponent: Normal range 1 to $2^E - 2$ (1 to 254), =true value+127 $(2^{E-1}-1)$.
Mantissa: D = denormalized mantissa value. M = 1.D $value = (-1)^{sign} \times 2^{Exp-127} \times M$, dynamic range = 254*6.02dB=1529dB

Value of denormalized number = $(-1)^S \times 2^{-(Bias-1)} \times D$

IEEE 754 single precision

	Exponent	Mantissa
Normalized numbers	1 to 2^E-2 (254), biased binary	Any number
Denormalized numbers	0	Non-zero
Zeroes	0	0
Infinities	2^E-1 (255)	0
NaNs	2^E-1 (255)	Non-zero

Addition: Adjust the smaller (absolute) number's exponent equal to the other exponent

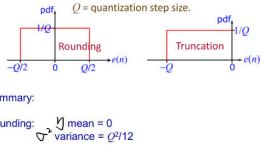
Quantizer

- Range of input signal = V_{min} to V_{max} (typically $V_{min} = -V_{max}$)
- Quantizer output = n bits

Typically, a quantizer is mid-tread (origin on tread of staircase)
- Number of quantizer levels = $2^n - 1$ $\to 2^{n-1}+1 \sim 2^{n-1}-1$
- Quantization step size $Q = \frac{V_{max} - V_{min}}{2^n - 1}$

A mid-rise quantizer (origin on rise of staircase)
- Number of quantizer levels = 2^n
- Quantization step size $\boxed{Q = \frac{V_{max} - V_{min}}{2^n}}$ $-2^{n-1} \sim 2^{n-1}$

Pdf: **probability density function** for e(n) is uniform, e(n) is a random variable. $[x(n)] = x(n) + e(n)$. expectation of x:$\eta = E\{g(x)\} = \int g(x)f(x)dx$. Variance of x:$\sigma^2$



Summary:

Rounding: mean = 0, variance = $Q^2/12$

Truncation: mean = $-Q/2$, variance = $Q^2/12$

$$SQNR = 10\log\left(\frac{\sigma_x^2}{\sigma_e^2}\right)$$
$$= 10\log\left(\frac{\sigma_x^2}{Q^2/12}\right)$$
$$= 10\log\left(\frac{3\sigma_x^2 2^{2n}}{V_{max}^2}\right)$$
$$= 6.02n + 4.77 + 20\log(\sigma_x/V_{max})$$

Auto-correlation of $e(n)$ is
$$R(m) = E\{e(n)e(n+m)\} = \begin{cases} E\{e(n)^2\} & m = 0 \\ 0 & m \neq 0 \end{cases}$$
But $E\{e(n)^2\} = \sigma^2$, variance of $e(n)$

Or, $R(m) = \sigma^2\,\delta(m)$

The probability density function for $e(n)$ is uniform from $-Q/2$ to $Q/2$ rounding
mean of $e(n) = 0$
maximum $e(n) = Q/2$
mean square error = variance of $e(n) = Q^2/12$
root mean square error = std dev of $e(n) = 0.29Q$ rms

Addition of uncorrelated noise sources:
- mean = $\eta_1 + \eta_2$
- variance = $\sigma_1^2 + \sigma_2^2$ R, S(ejw) no rms!

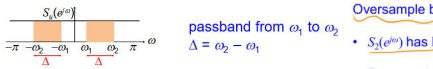
SQNR improves by 6 dB per additional bit of ADC; can also be improved by over-sampling and low-pass filtering. Doubling the sampling rate improves the SQNR by 3 dB.
Advantages and disadvantages of dithering.
Advantage: Adding noise provides more information; Actually, averaging of output samples leads to removal of noise and more information;
Disadvantage: Constant/slowly varying input signal is required for (averaging of) multiple similar samples; Otherwise, dithering increases output noise

Summary:

mean of output noise $\boxed{\eta_v = H(e^{j0})\eta_e}$ $\frac{2\Delta}{2\pi}$

variance of output noise $\sigma_v^2 = \left(\int_{-\pi}^{\pi} |H(e^{j\omega})|^2 \frac{d\omega}{2\pi}\right)\sigma_e^2$

input and output power spectral density are related by
$$S_{out}(e^{j\omega}) = |H(e^{j\omega})|^2 S_{in}(e^{j\omega})$$



passband from ω_1 to ω_2
$\Delta = \omega_2 - \omega_1$

Therefore, the mean of $v(n)$ is: $\eta_v = H(e^{j0})\eta_e$

Or, mean of $v(n)$ = gain of the system at ω=0 × mean of $e(n)$

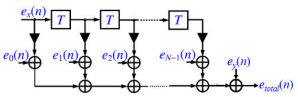
This result is true for any input and output:

$$S_{in}(e^{j\omega}) \to \boxed{H(e^{j\omega})} \to S_{out}(e^{j\omega}) = |H(e^{j\omega})|^2 S_{in}(e^{j\omega})$$

$$e(n) \to \boxed{H(e^{j\omega})} \to v(n)$$

Let $e(n) = \eta_e + u(n)$
→ $u(n)$ is a zero-mean white noise with variance $\sigma_u^2 = \sigma_e^2$ and power spectral density $S_u(e^{j\omega}) = \sigma_e^2$
mean = Nη; variance = Nσ^2;

Denote each mean by $\eta_x, \eta_0, \dots \eta_{N-1}, \eta_y, \eta_{total}$.
Then η_{total} is η_x filtered by $H(e^{j\omega})$, plus the means $\eta_0 \dots \eta_{N-1}, \eta_y$.
$$\eta_{total} = H(e^{j0})\eta_x + \eta_0 + \dots + \eta_{N-1} + \eta_y$$



Denote each variance by $\sigma_x^2, \sigma_0^2, \dots \sigma_{N-1}^2, \sigma_y^2, \sigma_{total}^2$.

Then σ_{total}^2 is σ_x^2 filtered by $H(e^{j\omega})$, plus the variances $\sigma_0^2, \dots \sigma_{N-1}^2, \sigma_y^2$.
$$\sigma_{total}^2 = \|H(e^{j\omega})\|_2^2 \sigma_x^2 + \sigma_0^2 + \dots + \sigma_{N-1}^2 + \sigma_y^2$$

If $e_x(n)$ is from quantization step size Q_x, $e_0(n)$ to $e_{N-1}(n)$ are from quantization step size Q, and $e_y(n)$ is from quantization step size Q_y, then
$$\sigma_{total}^2 = \|H(e^{j\omega})\|_2^2 \frac{Q_x^2}{12} + \frac{NQ^2}{12} + \frac{Q_y^2}{12}$$

$$S_{total}(e^{j\omega}) = |H(e^{j\omega})|^2 S_x(e^{j\omega}) + S_0(e^{j\omega}) + \dots + S_{N-1}(e^{j\omega}) + S_y(e^{j\omega})$$

ntization
$$S_{total}(e^{j\omega}) = |H(e^{j\omega})|^2 \frac{Q_x^2}{12} + \frac{NQ^2}{12} + \frac{Q_y^2}{12}$$

Error = input + output + N*multiplier

Nyquist sampling theorem

$x_c(t)$ is uniquely determined by its samples $x(n)$
$(x_c(t)$ can be perfectly reconstructed from its samples $x(n))$
if $f_s > 2B$

In general, for any integer m, sampling at
$$\frac{2f_c + B}{m+1} \le f_s \le \frac{2f_c - B}{m}$$
such that $f_s \ge 2B$, gives no aliasing

Amplifier: The gain is determined to match the input dynamic range to the ADC dynamic range; For unknown/time-varying input range, use an automatic gain controller (AGC) with time-varying gain;
Anti-aliasing filter: Analog low-pass filter with cut-off frequency B; Aliasing occurs if analog signal is not band limited to fs/2; Anti-aliasing filter limits the bandwidth of the analog signal;
N bit **Counter ADC**: $Q = V_{ref}/2^N$, 第一次比较≠0; Successive approximation ADC 二分式比较 N 次 from MSB to LSB;

Example: V_{ref} = 1 v (input range 0 to 1 v), V_{IN} = 0.51 v, 4-bit ADC $Q = 0.0625$

a_3 a_2 a_1 a_0	V_{IN}	DAC output	Comp
1/0/0 * * *	0.51 v	0.5 v	+ (a_3 = 1)
1 1/0 * *	0.51 v	0.75 v	– (a_2 = 0)
1 0 1/0 *	0.51 v	0.625 v	– (a_1 = 0)
1 0 0 1/0	0.51 v	0.5625 v	– (a_0 = 0)

Output = 1000
- Successive approximation ADC takes only n cycles
- Proceeds from MSB to LSB
- Popular because it is fast (2μsec for 12-bit ADC), accurate (high-resolution), low cost

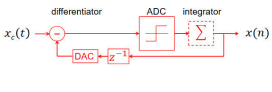
Dual slope integrating ADC: precise, insensitive to temperature and aging, slow, expensive;
Parallel (flash) ADC: very fast, more area, low resolution

$\frac{1-Q-V_{min}}{Q}$

Example: for −1v to 1v range, 4-bit ADC, V_{max} = 1−Q=0.882v, V_{min} =−1V,
$$Q = \frac{V_{max} - V_{min}}{2^4} = 0.118v$$
V_{ref} =1−(Q/2)=0.941v, ground = −1+(Q /2) = −0.941v, V_{IN} = 0.51v
parallel ADC
Output = 0100

V_{IN}	voltage divider	comp	priority encoder
0.51 v	6.5Q=0.765 v	+	找到:
0.51 v	5.5Q=0.647 v	+	最高位 0111 7
0.51 v	4.5Q=0.529 v	+	找到 1 000 8
0.51 v	3.5Q=0.412 v	–	
0.51 v	2.5Q=0.294 v	–	
0.51 v	1.5Q=0.176 v	–	
0.51 v	0.5Q=0.059 v	–	
0.51 v	−0.5Q=−0.059 v	–	$(0100)_{2's}$ = 4
0.51 v	−1.5Q=−0.176 v	–	
0.51 v	−2.5Q=−0.294 v	–	
0.51 v	−3.5Q=−0.412 v	–	
0.51 v	−4.5Q=−0.529 v	–	
0.51 v	−5.5Q=−0.647 v	–	
0.51 v	−6.5Q=−0.765 v	–	
0.51 v	−7.5Q=−0.882 v	–	

Sigma-delta ADC: reduce quantization error
1. Use differentiation



Analysis of differentiation of the signal:
- Auto-correlation of $x_c(t)$ is $R(\tau) = E\{x_c(t)x_c(t-\tau)\}$ $R(m)=E\{x[n]x[n-m]\}$
- Variance of $x_c(t)$ is $R(0) = \sigma_x^2$, variance of error is σ_1^2
- SQNR$_1$ $= 10\log(\sigma_x^2/\sigma_1^2) = 10\log(3\sigma_x^2 2^{2N}/V_{max}^2)$ $\sigma_1^2 = \frac{Q^2}{12}$, $= \frac{(V_{max}-V_{min})^2}{2^N}$

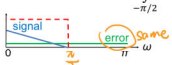
- Differentiation $d(t) = x_c(t) - x_c(t-T)$
- Variance of $d(t)$ is $\sigma_d^2 = E\{[x_c(t)-x_c(t-T)]^2\} = 2R(0)-2R(T)$
- Variance of error is σ_2^2, SQNR$_2$ $= 10\log(\sigma_d^2/\sigma_2^2)$

- If $\sigma_x/V_{max_x} = \sigma_d/V_{max_d}$ then SQNR$_1$ = SQNR$_2$, both quantizers perform equal, $\sigma_1/\sigma_1 = \sigma_d/\sigma_2$, or $\sigma_2 = \sigma_1(\sigma_d/\sigma_x)$
- If $R(T) > R(0)/2$ then $\sigma_d < \sigma_x$, the variance decreases = range of values decreases = better to quantize $d(t)$ $\sigma_d/\sigma_x<1$
- SQNR $10\log(\sigma_d^2/\sigma^2)$ increases by $10\log(\sigma_x^2/\sigma_d^2)$ dB

这段内容: condition for differentiation to work.
Prof. assume SQNR$_1$ = SQNR$_2$

Oversample by 2

- $S_2(e^{j\omega})$ has half width, double height 面积不变
- Same variance $\sigma_x^2 = \int_{-\pi/2}^{\pi/2} S_2(e^{j\omega})\frac{d\omega}{2\pi}$



- After lowpass filter, out-of-band error will be removed
real time
- 2 times sampling = σ_e^2 becomes half = 3dB more SQNR (each doubling of sampling gives 3dB more)
- Also, oversampling = lower T = higher $R(T)$ = lower σ_d^2 fs↑ T↓ R(T)↑ σ_d^2↓

3. integration/noise shaping

Error is reduced because order=1
- Highpass error $X_e(z) = (1 - z^{-1})E(z)$
- Magnitude of frequency response $|H(e^{j\omega})| = 2\sin\frac{\omega}{2}$ $\frac{1-e^{j\omega}}{2} = \sin\frac{\omega}{2}$
- $e(n)$ power spectral density = N white noise

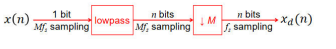
$|H(e^{j\omega})|^2 S_e(e^{j\omega})$

- $x_e(n)$ variance $\sigma_{x_e}^2 = \int_{-\pi}^{\pi} \left(2\sin\frac{\omega}{2}\right)^2 N\frac{d\omega}{2\pi} = 2N$
- $x_e(n)$ variance after lowpass filter
$$\sigma_{x_e}^2 = \int_{-B}^{B} \left(2\sin\frac{\omega}{2}\right)^2 N\frac{d\omega}{2\pi} \approx \int_{-B}^{B} \omega^2 N\frac{d\omega}{2\pi} = \frac{B^3 N}{3\pi}$$
- 2 times sampling = B becomes half = $\sigma_{x_e}^2$ becomes 1/8 = 9dB more SQNR

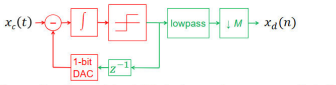
$B \to \frac{B}{2}$
$B^3 \to \frac{1}{8}B^3$

3dB from over-sampling
6dB from noise shaping

3) Decimator
Converts from 1-bit to n-bits



4) Block diagram of sigma-delta ADC



first-order sigma-delta ADC (red = analog, green = digital)

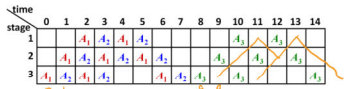
5) SQNR
Since any quantizer SQNR = 6.02n + .., adding 1 bit resolution = 6dB more SQNR.

For oversampling:
- Each doubling of sampling = 3dB more SQNR = adding ½ bit 6dB/6.02

For oversampling and first order noise shaping:
- Each doubling of sampling = 9dB more SQNR = adding 3/2 bits $\frac{9}{6.02}$

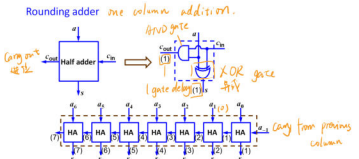
For oversampling and second order noise shaping:
- Each doubling of sampling = 15dB more SQNR = adding 5/2 bits $\frac{15}{6.02}$

Advantages of sigma-delta ADC: Low cost because 90% of silicon die area is digital processing; No need of high-precision analog component; High resolution, good noise characteristics; Anti-aliasing filter requirement is relaxed. **Disadvantages**: Low speed (due to oversampling), used for voice and audio; Slowly varying inputs cause low frequency artifacts.
DAC: Ladder: The resistor ladder network requires high precision component matching; Unity gain opamp acts as a buffer.
Zero-order: hold; first-order: interpolation.
Sigma-delta DAC: low-cost digital processing, simple analog filter

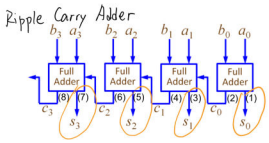


Initiation sequence: 1,7,1,7...
Average initiation = 4 clocks/sample
Average throughput = ¼ input/clock
larger smaller
latency =7 clocks 延迟

Forbidden set={0,2,4,6} Stage utilization = the fraction of time a given stage is being utilized; Maximum achievable (average) throughput≤ the reciprocal of the maximum number of utilized stages in a single row of the reservation table. 表中单行所用阶段数的倒数

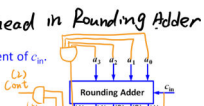


Each half adder needs 1 gate delay. The N-bit rounding adder needs N gate delays to complete. This is because the carry bit propagates from stage to stage. rippling carry 逐级进位



Carry Look-Ahead in Rounding Adder

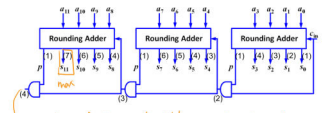
Let $p = a_3 \bullet a_2 \bullet a_1 \bullet a_0$.
p is computed independent of c_{in}.
$c_{out} = p \bullet c_{in}$



If each half adder, or each gate, requires 1 gate delay, then this block consisting of 4 (actually 3½) half adders requires 4 gate delays. However, p requires only 1 gate delay.

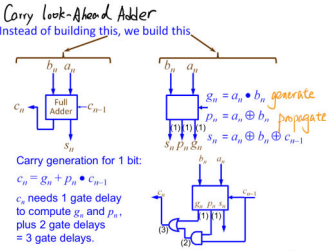
breaks the ripple

Why 3½. HA(4) only need an XOR gate, no need AND gate.



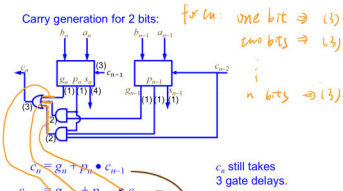
regardless of bits of Adder, only depends on number of cascading blocks.

Carry Look-Ahead Adder
Instead of building this, we build this



Carry generation for 1 bit:
$c_n = g_n + p_n \bullet c_{n-1}$
c_n needs 1 gate delay to compute g_n and p_n, plus 2 gate delays = 3 gate delays.

Carry generation for 2 bits:



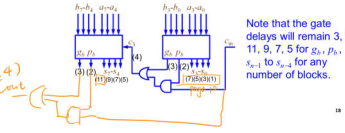
for c_n: one bit ⇒ (1)
two bits ⇒ (3)
⋮
n bits ⇒ (3)

$c_n = g_n + p_n \bullet c_{n-1}$
$c_{n-1} = g_{n-1} + p_{n-1} \bullet c_{n-2}$
$c_n = g_n + p_n \bullet \{g_{n-1} + p_{n-1} \bullet c_{n-2}\}$
$= g_n + p_n \bullet g_{n-1} + p_n \bullet p_{n-1} \bullet c_{n-2}$

c_n still takes 3 gate delays.

Block generate: $g_b = g_3 + p_3g_2 + p_3p_2g_1 + p_3p_2p_1g_0$

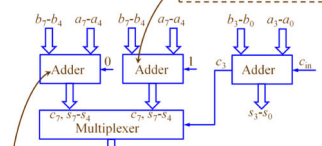
Block propagate: $p_b = p_3p_2p_1p_0$



Note that the gate delays will remain 3, 2, 11, 9, 7, 5 for g_b, p_b, s_{n-1} to s_{n-4} for any number of blocks.

Carry Select Adder

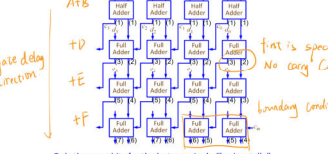
This adder computes c_7, and s_7 - s_4 assuming c_3 = 1.



This adder computes c_7, and s_7 - s_4 assuming c_3 = 0.

c_7 takes the time to compute c_3 plus one multiplexer (switching) delay.
much faster than gate delay

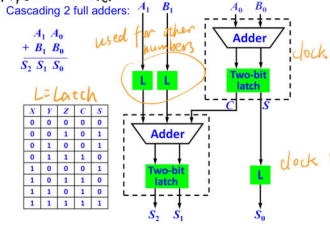
Carry Save Adder
Now consider the addition of 5 numbers.



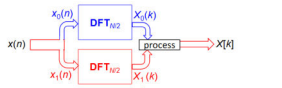
gate delay direction
first is special No carry C_{in}
bounding condition

Only the carry bits for the last row ripple "horizontally".
easy to add large number of bits; need switch to preserve bits

Pipelined Adder
Cascading 2 full adders:



Algorithm reduction:
DFT requires N^2 complex multiply and N(N–1) complex add. FFT: FFT requires about $(N/2)logN$ complex multiply and $NlogN$ complex add.



Each $N/2$ point DFT requires $N^2/4$ complex multiply and $N(N–2)/4$ complex add. The "process" requires N complex multiply and N complex add. Overall, the structure needs $N(N+2)/2$ complex multiply and $N^2/2$ complex add.

$$(a+jb)(A+jB) = (aA - bB) + j(aB + bA)$$

4 real multiply
2 real add

Rewrite it as:

$$(aA - bB + bA - bA) + j(aB + bA + bB - bB)$$
$$= \{(a+b)A - b(A+B)\} + j\{b(A+B) + (a-b)B\}$$

Let $c = a + b, d = a - b$ be precomputed, then:

3 real multiply

$$= \{cA - b(A+B)\} + j\{b(A+B) + dB\}$$

3 real add

Feed-forward cutset: cutset with all forward direction edges

Loop bound of a loop = $\frac{\text{loop computation time}}{\text{no of delays in the loop}}$

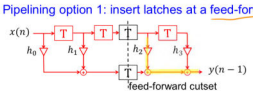
关键路径
Critical path of a graph is the longest computation time among all zero delay paths. $T_{critical} \le T_{clock}$
Clock cycle is lower bounded by the critical path.
should be as small as possible

Iteration bound is the maximum loop bound of a graph.
Critical loop is the loop with the maximum loop bound.

Pipeline: increase latency

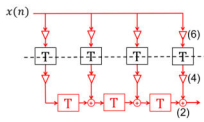
assume the same system

Pipelining option 1: insert latches at a feed-forward cutset



Pipelining option 2: fine-grain pipelining
Let T_M = 10 gate delays, T_A = 2 gate delays

To achieve a critical path less than T_M, break the multiplier into two parts with processing times of 6 gate delays and 4 gate delays.

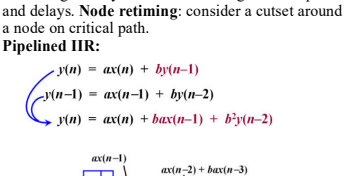


Critical path = 6 gate delays
Latency = 2
Throughput = 1

Option 3 **Retiming:** move around delays, reduce critical path and increase throughout, without increasing latency. Make cutset though critical path and delays. **Node retiming**: consider a cutset around a node on critical path.

Pipelined IIR:

$$y(n) = ax(n) + by(n-1)$$
$$y(n-1) = ax(n-1) + by(n-2)$$
$$y(n) = ax(n) + bax(n-1) + b^2y(n-2)$$



The input/output relationship for the above implementation is $y'(n) = ax(n-3) + bax(n-4) + b^2y'(n-2)$.

We have, $y(n) = ax(n) + by(n-1) = ax(n) + bax(n-1) + b^2y(n-2)$. Replacing n by $n-3$, we have $y(n-3) = ax(n-3) + bax(n-4) + b^2y(n-5)$.

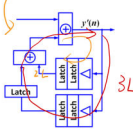
Latency = 4 (because of the latch), throughput =1

It can be seen that $y'(n) = y(n-3)$.

Example: Implement the filter
$y(n) = a_0y(n) + a_1x(n-1) + a_2x(n-2) + b_1y(n-1) + b_2y(n-2)$
using 2 latch multipliers and non-pipelined adders.

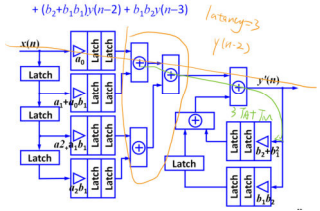
Solution: No $y(n-1)$

The structure of the feedback loop is as shown. There are two delays in the feedback path. Thus, it is a 2-latch pipelined system. Suppose the output of the 2-latch pipelined implementation is $y'(n)$. Since it is a 2-latch pipelined system, $y'(n-1)$ should be removed from the computation of $y'(n)$.



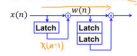
$$y(n) = a_0x(n) + (a_1 + a_0b_1)x(n-1) + (a_2 + a_1b_1)x(n-2) + a_2b_1x(n-3)$$
$$+ (b_2 + b_1b_1)y(n-2) + b_1b_2y(n-3)$$

latency=3
$y(n-2)$



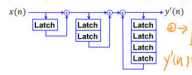
Example: Implement the filter $y(n) = x(n) + y(n-1)$ using 4-latch pipelined adders (no multiplier).

However, writing it as $(1 + z^{-1} + z^{-2} + z^{-3})X(z)$
$(1 + z^{-2})[(1 + z^{-1})X(z)] = (1 + z^{-2})W(z)$
where $W(z) = (1 + z^{-1})X(z)$, needs only 2 adders.



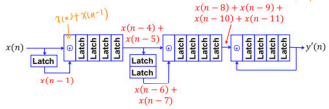
critical path only 2 adders

The complete implementation before pipelining is:



$y'(n) = y(n-12)$

The pipelined implementation is: all adders should be fine-pipelined



$x(n-8) + x(n-9) +$
$x(n-10) + x(n-11)$

The input/output relationship is:
$y'(n) = x(n-12) + x(n-13) + x(n-14) + x(n-15) + y'(n-4)$

Comparing with
$y(n) = x(n) + x(n-1) + x(n-2) + x(n-3) + y(n-4)$
it can be seen that $y'(n) = y(n-12)$.

Parallel processing: increase sampling rate/ throughout, no change critical path
Polyphase Parallel FIR Filter:

$$Y(z) = X(z)H(z), \text{where } X(z) = \sum x(n)z^{-n} \text{ etc.}$$

The input 2-phase polyphase components are

$$X_0(z) = \sum x(2n)z^{-n}, X_1(z) = \sum x(2n+1)z^{-n}$$

such that $X(z) = X_0(z^2) + z^{-1}X_1(z^2)$.

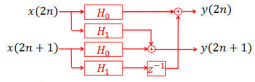
$X_0(z)$ is the z-transform of even-numbered inputs $x(2n)$.
$X_1(z)$ is the z-transform of odd-numbered inputs $x(2n+1)$.

It follows that: $Y_0(z) = X_0(z)H_0(z) + z^{-1}X_1(z)H_1(z)$
$Y_1(z) = X_0(z)H_1(z) + X_1(z)H_0(z)$

In matrix form (leaving z for brevity):

$$\begin{bmatrix} Y_0 \\ Y_1 \end{bmatrix} = \begin{bmatrix} H_0 & z^{-1}H_1 \\ H_1 & H_0 \end{bmatrix} \begin{bmatrix} X_0 \\ X_1 \end{bmatrix}$$

Implementation:



(Note that z^{-1} implies a delay of $2T$.)

The polyphase parallel FIR filter requires $2N$ multiply and $2(N–1)$ add with double the sampling frequency.
⇒ Strength reduction due to increased throughput.

Alternately, keeping the same sampling frequency, a slower multiplier/adder takes less area.
⇒ Strength reduction due to reduced area.

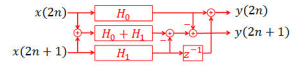
$$\begin{bmatrix} Y_0 \\ Y_1 \\ \vdots \\ Y_{L-1} \end{bmatrix} = \begin{bmatrix} H_0 & z^{-1}H_{L-1} & \cdots & z^{-1}H_1 \\ H_1 & H_0 & \cdots & z^{-1}H_2 \\ \vdots & \vdots & \ddots & \vdots \\ H_{L-1} & H_{L-2} & \cdots & H_0 \end{bmatrix} \begin{bmatrix} X_0 \\ X_1 \\ \vdots \\ X_{L-1} \end{bmatrix}$$

Low-Complexity Parallel FIR Filters (less adders and multipliers)

$$Y_1(z) = \{X_0(z) + X_1(z)\}\{H_0(z) + H_1(z)\} - X_0(z)H_0(z) - X_1(z)H_1(z)$$

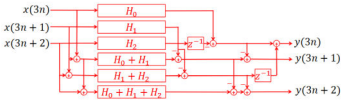
new computation
already computed for $Y_0(z)$

Implementation:



This low-complexity implementation needs only 3 subfilters, each of length $N/2$.
⇒ It requires $1.5N$ multiply and $1.5N+1$ add.

Implementation:



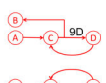
This structure needs 6 subfilters, each of length $N/3$.
⇒ It requires $2N$ multiply and $2N+4$ add.

Unfolding: disjoint loops 不相交环

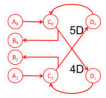
Step 2: For each edge $U \rightarrow V$ with m delays in the original data flow graph, draw J edges $U_i \rightarrow V_{(i+m)\%J}$ with $\left\lfloor \frac{i+m}{J} \right\rfloor$ delays for $i = 0, 1, \ldots, J-1$. if $m=0$ $U_i \rightarrow V_i$

(% = mod or modulo, find the remainder.)
($\lfloor \bullet \rfloor$ = floor, same as truncation.)

Edges without delay such as $A \rightarrow C$, $D \rightarrow C$ are unchanged.



Edge $C \rightarrow D$ has $m = 9$ delays.
So, $C_0 \rightarrow D_{(0+9)\%2} = D_1$ has $\lfloor (0+9)/2 \rfloor = 4$ delays.
$C_1 \rightarrow D_{(1+9)\%2} = D_0$ has $\lfloor (1+9)/2 \rfloor = 5$ delays.

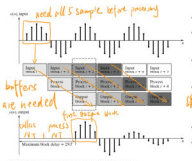


if 最大公约数 gcd(sum of delays, J) = 1, loops get connected

Unfolding a graph with iteration bound T_∞ results in the new iteration bound JT_∞. 迭代界限T∞迭代, J samples per clock. 信号频率J倍, 硬件上J倍复杂

Sampling Period (T)/ Instruction Cycle Time = # of Instructions per Sample

Block processing $N>5$



- Collect N samples at a time
- Processing must finish within N sample period, NT
- Normally used in FFT, Compression
- Requires double or triple buffering to perform acquisition and processing
- $T_p < N*T_s$ processing time
- A maximum block delay of $2NT_s$

Figure 1.17 Block processing of an input signal in a block of five samples

Double buffer Advantage: Can process more data for some given Ts due to the reduced setup time; Or use a higher Fs compared to the sampling approach.
Disadvantage: More memory required; More effort in programming; Processing latency

nx/nr = input length, nh = filter length how long your signal

C55x Unified Memory Map

Program and data share the same memory

Program memory: 16M x 8 bit, 24-bit address, *(00_0001h) = x

00_0000h	w
00_0001h	x
00_0002h	y
00_0003h	z
FF_FFFFh	s

Data memory: 8M x 16 bit, 23-bit address, *(00_0001h)=yz

segmented into 128 pages (7 msb bits)

page = 64K words (16 lsb bits)

Accumulators: AC0 to AC3 keep adding
Located in D unit, used for ALU/MAC/shifter output, each portion can be accessed individually or together

guard=bits 39-32	high = bits 31-16	low = bits 15-0

Examples:
AC0 40 bits
HI(AC0) 16 bits (31-16)
LO(AC0) 16 bits (15-0)

Auxiliary registers: AR0 to AR7
Located in A unit, used to address data memory,
ARxH = 7-bit memory page, ARx = 16 lsb bits of address within a page

ARxH = bits 22-16	ARx = bits 15-0

XARx = bits 22-0

Examples:
AR0 16 bits (15-0)
XAR0 23 bits

MOV AR0, AR1 move 16 bits from AR0 to AR1
MOV XAR0, XAR1 move 23 bits from XAR0 to AR1

x < y x bits sign extended to y bits, 符号位扩展0/1 ←value
except when x = 23 bits, x bits zero filled to y bits ←address

Temporary registers: T0 to T3
Located in A unit, used for multiplicand, shift count, pointer value, transition matrix, etc.

Tx = bits 15-0

Example:
T0 16 bits

Tx works in the exact same way as ARx shown before

Pair of registers: AR0, AR2, AR4, AR6, T0, T2
Refers to 2 registers such as AR0 and AR1

Example:
Pair(AR0) 2 x 16 bits

Value
Denoted by "#"
Can be in decimal or hex (denoted by "h")

k4/-k4: 4-bit unsigned constant (with "-" if negative) range −15 ~ +15

K8: 8-bit signed constant **K16**: 16-bit signed constant

Value → register uses k4, -k4, K16
Value → memory uses K8, K16

Direct addressing:
Denoted by "@"
(assume compiler mode bit CPL=1 in status register)
@Daddr = Use 7-bit DPH + 16-bit (DP + Doffset), then 16-bit data at this address. Doffset = 7 lsb bits of (Daddr – DP).

Daddr = bits 15-0 @65324 = FFFAh
− DP = bits 15-0 FFF0h
= neglect Doffset = bits 6-0 2Ah
if neglect = 0 Daddr no change

DP = bits 15-0 FFF0h
+ Doffset = bits 6-0 2Ah
= address bits 15-0 = 02_FFFA
DPH = bits 22-16 02
address bits 22-0

Auxiliary register (AR) indirect addressing:
Denoted by "*"
*ARx = 16-bit data at XARx

XARx = bits 22-0

Example:
*AR0 16-bit data at XAR0
low_byte(*AR0) 8-bit (bit 7-0) data at XAR0
high_byte(*AR0) 8-bit (bit 15-8) data at XAR0
*AR0, *AR1 2x16-bit data, bit 15-0 at XAR0, bit 31-16 at XAR1
dbl(*AR0) 32-bit data. If XAR0 = even, then bit 31-16 at XAR0 and bit 15-0 at XAR0+1. If XAR0 = odd, then bit 31-16 at XAR0 and bit 15-0 at XAR0-1.
double

Operands
Operands for AR indirect addressing:
*ARx no modification

*ARx± increase/decrease ARx after addressing
*±ARx increase/decrease ARx before addressing

*(ARx±AR0) add/subtract AR0 to ARx after addressing
*(ARx±Ty) add/subtract Ty to ARx after addressing

*ARx(offset) no modification to ARx, but offset is used while addressing. offset = AR0, Ty, #K16

*+ARx(#K16) add #K16 to ARx before addressing

MAC m1, m2, sum
sum = sum + (m1 * m2)

- sum = ACx 40bits
- m1, m2 of shorter length are sign extended to 17 bits ≤16 bits
- 32-bit multiplication result is sign extended to 40 bits and added to ACx. If an overflow is detected, accumulator overflow status bit is set.
- Rounding and/or shifting may be performed if needed.

There are 4 types of MAC based on the choice of m1, m2

1) **MAC** with registers: m1 = ACx, m2 = Ty 16bits
ACx: only bits 32-16 are multiplied 17 bits
Ty: sign extended to 17 bits

Example:
MAC AC0, T1, AC2 AC2 = AC2 + ($AC0_{32-16}$ * T1)

2) **MACK** with value: m1 = Tx, m2 = K8/K16
K8/K16: 8/16-bit signed value, sign extended to 17 bits

3) **MACM** with memory: m1 = memory, m2 = ACx/Tx/memory
memory: sign extended to 17 bits

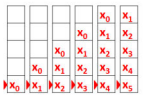
Examples:
MACM *AR0, AC0, AC1 AC1 = AC1 + (data * $AC0_{32-16}$)
MACM *AR0, *AR1, AC0 AC0 = AC0 + (data1 * data2)

4) **MACMK** with memory and value: m1 = memory, m2 = K8
Example:
MACMK *AR0, #FFh, AC0 AC0 = AC0 + (data * FFh)

Parallel instructions: MAC:: MAC, MOV||MOV

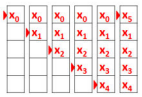
Circular buffer addressing
C55x permits circular buffer for filtering

Linear buffer



Shift all past values

Circular buffer



Shift the pointer
Pointer is incremented modulo the buffer size

A linear/circular configuration bit in status register controls whether a pointer is linear or circular.

A circular buffer consists of:
- One buffer start address register
- One buffer size register
- Pointer offset stored in ARx or CDP

It is possible to have up to 3 simultaneous circular buffers of different sizes.

CPU: Instruction buffer (I) unit; Program flow (P) unit; Address-data flow (A) unit; Data computation (D) unit

Saturation
During overflow, the result may be saturated.

- If saturation mode bit for D unit (SATD) is 1, then positive result is typically saturated to 00_7FFF_FFFFh and negative result is saturated to FF_8000_0000h
- If saturation mode bit for A unit (SATA) is 1, then positive saturated to 7FFFh and negative to 8000h
- If saturation on multiplication bit (SMUL) is 1, then same as SATD for multiplication result

In the previous example, with saturation,
5000h + 3000h = 7FFFh = 0.99997
C000h + B000h = 8000h = −1

If fractional mode status bit (FRCT) is 1, products are automatically shifted left by 1 bit.

Overflow gives wrap around effect for 2's complement numbers: A large positive number becomes a large negative number, and vice versa.
Saturation clips the error: A large positive number becomes a slightly smaller positive number, and vice versa. Therefore, saturation introduces less error.

TMS320C55x example:

Function	Description	Benchmarks
fir2	Finite Impulse Response Filter	Core: nx * (3+nh/2); Overhead: 25
dlms	Adaptive Delayed LMS Filter	Core: nx * (7+2*(nh-1)) Overhead: 26
convol	Convolution	Core: nr * (1+nh) Overhead: 44
cfft	Complex FFT	(FFT Size,Cycle) = (8,208) (16,358) ... (512,11848) (1024,25954)
rand16, rand16init	Random Number Generation (16-bit) and Its Initialization	rand16: Core: 13 + nr*2, Overhead:10; rand16init: 6

pipeline
overall = core + overhead
nr/2

nx/nr = input length, nh = filter length how long your signal